Müsahibə sualları

1.Var Object Dynamic nədir?

`var` tip deyil, compiler tipi özü çıxarır.

var name = "Ali";

Compiler bunu belə yazılmış kimi qəbul edir:

string name = "Ali";

Buna görə:

var name = "Ali";

name.Length; // OK

name = 5; // Compile Error

`var` ayrıca tip deyil, compiler üçün yazılışı sadələşdirən syntactic sugar**-dır. Compile-time zamanı compiler sağ tərəfdəki ifadəyə baxaraq real tipi müəyyən edir və kodu həmin tip kimi kompilyasiya edir.**

`object` .NET-də bütün tiplərin baza tipidir.

object obj = "Ali";

Compiler yalnız `object` görür:

obj.Length; // Compile Error

Çünki `object` tipində `Length` yoxdur.

Cast etmək lazımdır:

object obj = "Ali";

Console.WriteLine(((string)obj).Length);

`dynamic` tip yoxlamasını runtime-a saxlayır.

dynamic d = "Ali";

Console.WriteLine(d.Length); // OK

Compiler yoxlamır.

dynamic d = "Ali";

d.Test(); // **Compile olunur**

Amma işləyəndə: RuntimeBinderException

#Value type-lar həmişə stack-da saxlanılmır. Əgər value type bir `class` daxilində field kimi istifadə olunursa, həmin field heap-də yerləşən obyektin daxilində inline saxlanılır. Bunun səbəbi `struct`-ların dəyəri özündə saxlaması və ayrıca heap obyekti yaratmamasıdır. Yəni stack/heap qərarı tipdən yox, obyektin harada yerləşməsindən asılıdır.

2. record class ilə record struct fərqləri: bərabərlik semantikasına və yaddaş davranışına təsiri nədir?

Həm `record class`, həm də `record struct` **value-based equality** təmin edir, yəni müqayisə referensə görə deyil, property-lərin dəyərlərinə görə aparılır. Əsas fərq onların reference type və value type olmasındadır.

3.StringPool İntern() davranışı

String Pool-da eyni məzmunlu string-in olması o demək deyil ki, Heap-də yaradılan bütün string obyektləri avtomatik həmin referansı istifadə edəcək.

`new string(...)` həmişə yeni obyekt yaradır.

`string.Intern()` isə mövcud referansı Pool-dakı obyektlə əvəz edir.

Müsahibədə soruşsalar, bu cümlə çox güclü cavabdır:

"`new string(...)` intern pool-u bypass edir və həmişə yeni obyekt yaradır. `string.Intern()` isə həmin obyektin məzmununa uyğun intern edilmiş instansiyanı qaytarır; obyekti pool-a köçürmür, sadəcə pool-dakı referansı qaytarır."

4.StringBuilder davranışı

StringBuilder daxildə mutable char buffer (char array) saxlayır. String immutable olduğu üçün hər dəyişiklikdə yeni obyekt yaradılır. StringBuilder isə mövcud buffer üzərində dəyişiklik etdiyi üçün çoxlu concatenation əməliyyatlarında daha performanslıdır.

```
var sb = new StringBuilder();
```

sb.Append("Hello");

sb.Append("World");

1. İlk vəziyyət

buffer: [_ _ _ _ _ _ _ _]

capacity: 16 (default)

length: 0

2. "Hello" əlavə olunur

sb.Append("Hello");

buffer: [H e l l o _ _ _ _ _]

length: 5

👉 Yeni string yaradılmadı
👉 Sadəcə array içi dolduruldu

buffer: [H e l l o W o r l d _]

length: 10

5. Niyə Dispose() içində GC.SuppressFinalize(this) yazırıq?

Çünki obyekt artıq resource-ları təmizləyib. Finalizer-in sonradan işləməsinə ehtiyac yoxdur. `GC.SuppressFinalize(this)` obyekti finalization queue-dan çıxarır və GC-nin əlavə iş görməsinin qarşısını alır.

6.Amortized O(1) nədir?

Bu anlayış ən çox List<T>.Add() üçün istifadə olunur.

Məsələn:

Başlanğıc kapasite:

```
4
```

```
[10][20][30][40]
```

Doludur.

İndi:

```C#
list.Add(50);
```

Nə baş verir?

Yeni massiv yaradılır:

8 ölçülü

[10][20][30][40][ ][ ][ ][ ]

Köhnə elementlər köçürülür.

Bu əməliyyat:

O(n)

-dir.

Bəs niyə Add() O(1) sayılır?

Çünki bu köçürmə hər əlavədə olmur.

Kapasite belə artır:

4

8

16

32

64

128

...

Yəni nadir hallarda böyük köçürmə edilir.

Minlərlə əlavə üçün ümumi xərc

O(n)

olur.

Deməli bir əlavəyə düşən orta xərc:

O(1)

Bu da amortized O(1) adlanır.

Nə vaxt real O(n) olur?

Məhz kapasite dolanda:

list.Add(item);

aşağıdakılar baş verir:

1. Yeni array yarat
2. Bütün elementləri köçür
3. Yeni elementi əlavə et

Bu konkret əməliyyat:

O(n)

-dir.